

OptiCrop: Smart Agricultural Production Optimization Engine

Project Description

OptiCrop harnesses the power of **Machine Learning** to provide intelligent crop recommendations, enabling farmers and agricultural stakeholders to make data-driven farming decisions. The platform includes a **Crop Recommendation Module**, which predicts the most suitable crop based on soil nutrients and environmental conditions, a **Data Analysis Module**, which analyzes agricultural datasets to identify crop-environment relationships, and a **Prediction Engine**, which delivers accurate crop recommendations using a trained Random Forest machine learning model.

Utilizing **Scikit-learn** with the **Random Forest algorithm**, OptiCrop processes user-provided soil and climate parameters such as **Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH, and rainfall** to generate fast and reliable crop recommendations. Built with the **Flask web framework** and an interactive **HTML, CSS, and Bootstrap** user interface, the application provides a seamless and user-friendly experience. The trained model is stored using **Pickle (.pkl)**, ensuring efficient prediction and quick response times. OptiCrop empowers farmers, agricultural researchers, agribusiness organizations, and policymakers to improve crop productivity, optimize resource utilization, and promote sustainable agricultural practices.

Scenarios

Scenario 1

A farmer enters soil nutrient values (Nitrogen, Phosphorous, and Potassium) along with environmental conditions such as temperature, humidity, pH level, and rainfall. The system analyzes the inputs and instantly recommends the most suitable crop for achieving higher yield and efficient farming.

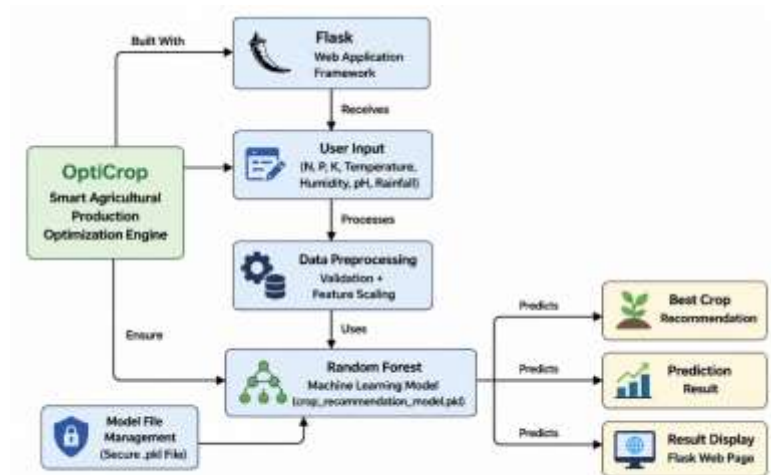
Scenario 2

An agricultural researcher evaluates different combinations of soil and climate conditions to study crop suitability. OptiCrop predicts the most appropriate crop and helps identify patterns that support agricultural research and sustainable farming strategies.

Scenario 3

An agribusiness consultant or policymaker uses OptiCrop to assess crop suitability across different environmental conditions. The platform provides intelligent crop recommendations that assist in resource planning, improving agricultural productivity, and supporting evidence-based farming decisions.

Technical Architecture:



Description:

OptiCrop utilizes a modular three-tier architecture to provide intelligent crop recommendation using machine learning. The frontend offers a responsive and user-friendly interface developed with HTML, CSS, and Bootstrap, allowing users to enter soil nutrient and environmental parameters. The Flask-based backend performs input validation, data preprocessing, and prediction by loading a trained Random Forest machine learning model stored in Pickle (.pkl) format. The application processes parameters such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH, and rainfall to recommend the most suitable crop. The system is developed locally using Flask and can be deployed on cloud platforms such as Render **or executed through the Flask development server for real-time crop prediction.**

Pre-requisites:

1. **Python Programming Knowledge:** Python Documentation
2. **Flask Framework Knowledge:** Flask Documentation
3. **Machine Learning Fundamentals:** Scikit-learn Documentation
4. **NumPy and Pandas for Data Processing:** NumPy & Pandas Documentation
5. **Data Visualization:** Matplotlib & Seaborn Documentation
6. **HTML, CSS, and Bootstrap Skills:** W3Schools HTML/CSS/Bootstrap Tutorials

7. **Jupyter Notebook / Anaconda Environment:** Anaconda Documentation
8. **Version Control with Git and GitHub:** Git Documentation
9. **Flask Application Deployment (Local/Render):** Flask Deployment Guide

Project Workflow

Milestone 1: Environment Setup, Model Selection and Architecture

- **Activity 1.1:** Set up the Python virtual environment and install required libraries such as Flask, Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn, and Pickle.
- **Activity 1.2:** Analyze the agricultural dataset and compare machine learning algorithms including K-Nearest Neighbors (KNN), Logistic Regression, Decision Tree, Random Forest, and K-Means Clustering.
- **Activity 1.3:** Select the best-performing machine learning model (Random Forest) based on evaluation metrics.
- **Activity 1.4:** Design the overall application architecture, defining the interaction between the Flask frontend, backend, preprocessing module, and machine learning model.

Milestone 2: Machine Learning Model Development

- **Activity 2.1:** Preprocess the agricultural dataset by handling missing values, feature scaling, and data cleaning.
- **Activity 2.2:** Train and evaluate multiple machine learning models using soil nutrients (N, P, K), temperature, humidity, pH, and rainfall.
- **Activity 2.3:** Save the trained Random Forest model and feature scaler as **.pkl** files using Pickle for deployment.

Milestone 3: Backend Development (Flask)

- **Activity 3.1:** Develop the Flask backend and implement routing for the application.
- **Activity 3.2:** Create modules for user input validation, data preprocessing, and crop prediction.
- **Activity 3.3:** Integrate the trained machine learning model with the Flask application to generate crop recommendations.

Milestone 4: Frontend Development

- **Activity 4.1:** Design a responsive and user-friendly interface using HTML, CSS, Bootstrap, and JavaScript.
- **Activity 4.2:** Create input forms for Nitrogen (N), Phosphorous (P), Potassium (K), Temperature, Humidity, pH, and Rainfall.
- **Activity 4.3:** Display crop prediction results dynamically using Flask templates.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Perform application testing using Postman and browser-based testing to validate prediction accuracy and functionality.
- **Activity 5.2:** Configure the project for deployment by installing dependencies and setting up the Flask server.
- **Activity 5.3:** Deploy the OptiCrop application on a cloud hosting platform such as **Render** for public access.

Milestone 6: Conclusion

- Evaluate the overall system performance and prediction accuracy.
- Document project outcomes, limitations, and future enhancements.
- Deliver a machine learning-based crop recommendation system that assists farmers in selecting suitable crops based on soil and environmental conditions.

Milestone 1.

Milestone 1: Model Selection and Architecture

In this milestone, the primary focus is on preparing the agricultural dataset, selecting the most suitable machine learning algorithm, and designing the overall architecture of the OptiCrop system. Multiple machine learning algorithms are analyzed and compared based on their prediction accuracy and performance. After evaluation, the **Random Forest** algorithm is selected as the final model due to its high accuracy, robustness, and ability to handle agricultural datasets effectively. The application architecture is then designed to integrate the Flask web application with the trained machine learning model, enabling real-time crop recommendations.

Activity 1.1: Environment Setup and Library Installation

Before developing the OptiCrop application, configure the Python development environment and install all required libraries.

Install Python and Create a Virtual Environment

 **Bash**

```
python -m venv optienv
```

Windows

</> Bash

```
optienv\Scripts\activate
```

Install all required packages.

</> Bash

```
pip install -r requirements.txt
```

</> Bash

```
pip install flask
pip install scikit-learn
pip install pandas
pip install numpy
pip install matplotlib
pip install seaborn
pip install pickle-mixin
pip install jupyter
```

Activity 1.2: Dataset Analysis and Model Selection

The OptiCrop system uses an agricultural dataset containing soil nutrients and environmental parameters.

Dataset Features

- Nitrogen (N)
- Phosphorous (P)
- Potassium (K)
- Temperature
- Humidity
- pH
- Rainfall

Output

- Recommended Crop

Steps Performed

- Load the dataset using Pandas.
- Perform data cleaning and preprocessing.
- Handle missing values (if any).
- Perform Exploratory Data Analysis (EDA).
- Split the dataset into training and testing sets.
- Train multiple machine learning algorithms.

Algorithms evaluated include:

- K-Nearest Neighbors (KNN)
- Logistic Regression
- Decision Tree
- Random Forest
- K-Means Clustering

After comparing prediction accuracy and evaluation metrics, **Random Forest** is selected as the final model because it provides the highest prediction accuracy and stable performance for crop recommendation.

Activity 1.3: Define the Application Architecture

Create an architectural diagram representing the interaction between different components of the OptiCrop system.

The architecture consists of:

- User Interface developed using HTML, CSS, and Bootstrap.
- Flask Backend for request handling and prediction processing.
- Data Preprocessing module for validating and scaling input values.
- Trained Random Forest Model for crop prediction.
- Prediction Result module for displaying the recommended crop to the user.

The workflow is as follows:

User → Flask Application → Data Preprocessing → Random Forest Model → Crop Recommendation → Result Display

Activity 1.4: Development Environment Configuration

Create the project directory structure.

```
OptiCrop/
|
|— app.py
|— requirements.txt
|— README.md
|— train_model.py
|— dataset.csv
|
|— models/
|   |— crop_recommendation_model.pkl
|   |— scaler.pkl
|
|— templates/
|   |— index.html
|
|— static/
|   |— css/
|   |— js/
|   |— images/
|
|— notebooks/
```

Required Python Libraries

- Flask
- Scikit-learn
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Pickle

Development Tools

- Python 3.10+
- Jupyter Notebook
- Visual Studio Code
- Git & GitHub

Milestone 1 Outcome

At the end of this milestone:

- The development environment is successfully configured.
- The agricultural dataset is analyzed and preprocessed.
- Multiple machine learning algorithms are trained and evaluated.
- The Random Forest model is selected as the final prediction model.
- The application architecture is finalized.
- The trained model is saved in **.pkl** format for integration with the Flask web application.

Milestone 2: Core Functionalities Development

In this milestone, the core functionalities of the OptiCrop application are developed. The focus is on preprocessing agricultural data, training the machine learning model, integrating the trained model with the Flask backend, and generating intelligent crop recommendations based on user-provided soil and environmental parameters.

Activity 2.1: Develop the Crop Recommendation Model

Implement the machine learning model responsible for predicting the most suitable crop.

Load the Agricultural Dataset

- Read the agricultural dataset using **Pandas**.
- Verify the dataset for missing values and inconsistencies.
- Explore the dataset using summary statistics and visualization techniques.

Data Preprocessing

Perform the following preprocessing steps before training:

- Remove duplicate records.
- Handle missing values (if any).
- Separate input features and target labels.
- Apply feature scaling using **StandardScaler**.
- Split the dataset into training and testing sets.

Train Multiple Machine Learning Models

Train and compare different machine learning algorithms:

- K-Nearest Neighbors (KNN)
- Logistic Regression
- Decision Tree
- Random Forest
- K-Means Clustering

Model Evaluation

Evaluate each model using:

- Accuracy Score
- Confusion Matrix
- Classification Report
- Cross Validation

After evaluation, select the **Random Forest** model because it achieved the highest prediction accuracy.

Save the Model

Store the trained model and scaler using Pickle for future predictions.

 Python

```
import pickle

pickle.dump(model, open("models/crop_recommendation_model.pkl", "wb"))
pickle.dump(scaler, open("models/scaler.pkl", "wb"))
```

Activity 2.2: Implement the Flask Backend

Develop the Flask backend to process user requests and interact with the trained machine learning model.

Define Flask Routes

Create the following routes in **app.py**:

- `/` → Displays the OptiCrop homepage.
- `/predict` → Receives user inputs and returns the predicted crop.

Example:

`</>` Python

```
@app.route("/")
def home():
    return render_template("index.html")
```

`</>` Python

```
@app.route("/predict", methods=["POST"])
def predict():
```

Process User Input

Accept the following parameters from the user:

- Nitrogen (N)
- Phosphorous (P)
- Potassium (K)
- Temperature
- Humidity
- pH
- Rainfall

Validate the inputs to ensure:

- No empty values
- Numeric values only
- Values fall within acceptable agricultural ranges

Data Preprocessing

Before prediction:

- Convert input values into numerical format.
- Scale the features using the saved StandardScaler.
- Arrange the values in the correct feature order.

Example workflow:

```
User Input
  ↓
Input Validation
  ↓
Feature Scaling
  ↓
Random Forest Model
  ↓
Predicted Crop
```

Generate Crop Recommendation

Load the trained Random Forest model.

 Python

```
model = pickle.load(open("models/crop_recommendation_model.pkl", "rb"))
```

Pass the processed input to the model.

```
</> Python  
  
prediction = model.predict(input_data)
```

Return the predicted crop to the frontend.

Error Handling

Implement exception handling for:

- Invalid user inputs
- Missing values
- Prediction failures
- Unexpected server errors

```
</> Python  
  
try:  
    prediction = model.predict(input_data)  
except Exception:  
    return "Prediction Failed"
```

Milestone 2 Outcome

At the end of this milestone:

- Agricultural data is successfully preprocessed.
- Multiple machine learning models are trained and evaluated.
- The Random Forest model is integrated with the Flask backend.
- User inputs are validated before prediction.

- The application generates accurate crop recommendations based on soil nutrients and environmental conditions.
- Error handling ensures stable and reliable application performance.

Milestone 3: App.py Development

Milestone 3 focuses on developing the core logic of the **app.py** file, which acts as the backbone of the OptiCrop application. In this milestone, Flask routes are created to manage user requests, process agricultural input values, perform data preprocessing, load the trained machine learning model, and generate crop recommendations. Proper input validation and exception handling are also implemented to ensure reliable system performance.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py

Establish routes for the OptiCrop application, each serving a specific purpose.

Home Route (/)

Displays the main user interface where users can enter soil nutrients and environmental parameters.

```
</> Python

@app.route("/")
def home():
    return render_template("index.html")
```

Prediction Route (/predict)

Accepts user input, validates the values, processes the data, loads the trained Random Forest model, and returns the recommended crop.

```
@app.route("/predict", methods=["POST"])
def predict():
    try:
        n = float(request.form["N"])
        p = float(request.form["P"])
        k = float(request.form["K"])
        temperature = float(request.form["temperature"])
        humidity = float(request.form["humidity"])
        ph = float(request.form["ph"])
        rainfall = float(request.form["rainfall"])

        input_data = [[n, p, k, temperature, humidity, ph, rainfall]]

        input_scaled = scaler.transform(input_data)

        prediction = model.predict(input_scaled)
```

```
    return render_template(
        "index.html",
        prediction_text=f"Recommended Crop: {prediction[0]}"
    )

except Exception:
    return render_template(
        "index.html",
        prediction_text="Invalid input values."
    )
```

Set Up Route Handlers

For the **/predict** route, implement logic to read the following agricultural parameters submitted by the user:

- Nitrogen (N)
- Phosphorous (P)
- Potassium (K)
- Temperature
- Humidity

- pH
- Rainfall

The application validates all user inputs before passing them to the prediction model.

Validation includes:

- No empty fields
- Numeric values only
- Valid agricultural ranges
- Error handling for invalid inputs

If validation fails, an appropriate error message is displayed to the user.

Load the Trained Machine Learning Model

Load the trained Random Forest model and Standard Scaler during application startup.

```
</> Python

import pickle

model = pickle.load(open("models/crop_recommendation_model.pkl", "rb"))
scaler = pickle.load(open("models/scaler.pkl", "rb"))
```

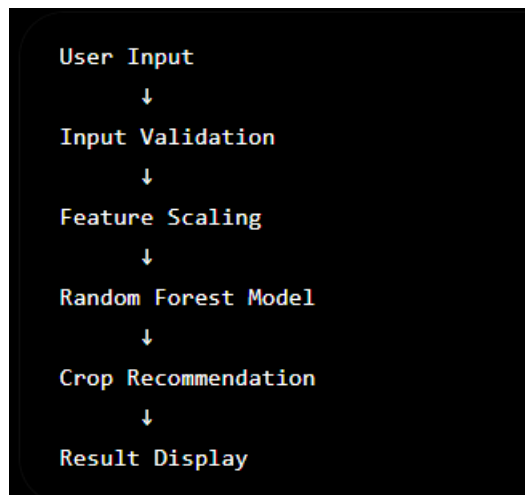
These files are generated during the model training phase and reused for real-time prediction.

Data Preprocessing

Before prediction, the application preprocesses user inputs by:

- Converting all values into numeric format.
- Arranging features in the correct order.
- Applying feature scaling using the saved Standard Scaler.

Processing workflow:

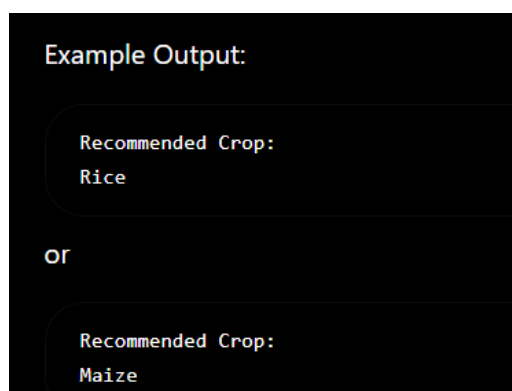


Generate Crop Recommendation

The preprocessed input is passed to the trained Random Forest model.

```
</> Python  
  
prediction = model.predict(input_scaled)
```

The predicted crop is returned and displayed on the result page.



Error Handling

Implement exception handling to improve application reliability.

The application handles:

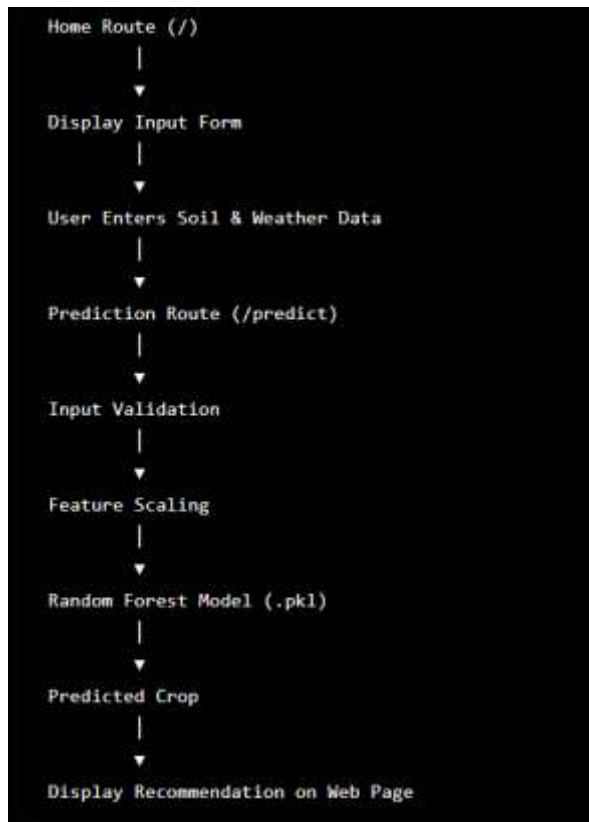
- Invalid numeric inputs
- Missing field values
- Prediction failures
- Unexpected runtime errors

Example:

```
</> Python

try:
    prediction = model.predict(input_scaled)
except Exception:
    return render_template(
        "index.html",
        prediction_text="Prediction Failed. Please enter valid values."
    )
```

Application WorkFlow



Milestone 3 Outcome

At the end of this milestone:

- Flask routes are successfully implemented.
- The trained Random Forest model is integrated into the application.
- User inputs are validated before prediction.
- Agricultural data is preprocessed automatically.
- Crop recommendations are generated accurately in real time.
- Proper exception handling ensures stable and reliable application performance.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a responsive, interactive, and user-friendly web interface for the OptiCrop application. The frontend enables users to enter soil nutrient and environmental parameters, submit prediction requests, and view crop recommendations in an organized and visually appealing dashboard. HTML, CSS, Bootstrap, and JavaScript are used to create a modern interface that enhances usability and accessibility.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure

- Develop the main HTML file (**index.html**) as the primary interface for the OptiCrop application.
- Create a header containing the **OptiCrop** project title, logo, and application description.
- Design separate sections for:
 - Project Overview
 - Agricultural Input Form
 - Crop Recommendation Result
 - Information Cards explaining how the prediction system works.
- Use semantic HTML elements to improve readability, maintainability, and accessibility.
- Integrate **Bootstrap** components and **Font Awesome** icons to enhance the appearance of the interface.

Design a Responsive Layout Using CSS

- Create a stylesheet (**static/css/style.css**) to provide a clean and modern user interface.
- Apply responsive design principles using **Bootstrap Grid** and **Flexbox** layouts.
- Organize the dashboard into multiple sections including:

- Project Information
 - Machine Learning Model Details
 - Agricultural Parameter Input Form
 - Prediction Result
- Use media queries to ensure compatibility across desktop, tablet, and mobile devices.

Create Separate UI Components

Develop individual interface components for different functionalities.

Project Dashboard

Display:

- Project Title
- Application Description
- Machine Learning Model Used
- Model Accuracy
- Number of Crop Classes

Input Form

Create input fields for:

- Nitrogen (N)
- Phosphorous (P)
- Potassium (K)
- Temperature

- Humidity
- pH
- Rainfall

Provide placeholder text and validation messages for each field.

Prediction Result Card

Display the predicted crop in a highlighted result section after successful prediction.

Example:

Recommended Crop

Rice

Information Section

Display informational cards describing:

- Smart Data Preprocessing
- Machine Learning Prediction
- Decision Support
- Adaptive Crop Recommendations

These cards help users understand how the OptiCrop system works.

Activity 4.2: Creating Dynamic Content Rendering

Handle Form Submission

- Implement the prediction form using HTML forms.
- Accept user inputs for all agricultural parameters.
- Submit the data securely to the **/predict** route using the POST method.

Display Prediction Results

- After processing the input values, dynamically display the recommended crop on the webpage.
- Present the result in a clearly visible card or alert box.
- If invalid data is entered, display an appropriate error message without crashing the application.

Improve User Experience

Implement additional frontend features such as:

- Clear placeholder text for all input fields.
- Responsive buttons with hover effects.
- Input validation before form submission.
- Attractive Bootstrap cards and icons.
- Easy-to-read typography and spacing.
- Smooth page navigation between the input and result sections.

Responsive Design

Ensure that the OptiCrop interface functions correctly on:

- Desktop Computers
- Laptops

- Tablets
- Mobile Devices

The interface automatically adjusts component sizes and layouts according to screen resolution.

Milestone 4 Outcome

At the end of this milestone:

- A responsive and user-friendly web interface is successfully developed.
- Users can enter soil and environmental parameters through an intuitive input form.
- The application displays crop recommendations dynamically after prediction.
- Bootstrap and CSS provide a clean and professional dashboard layout.
- The frontend integrates seamlessly with the Flask backend, ensuring a smooth user experience across different devices.

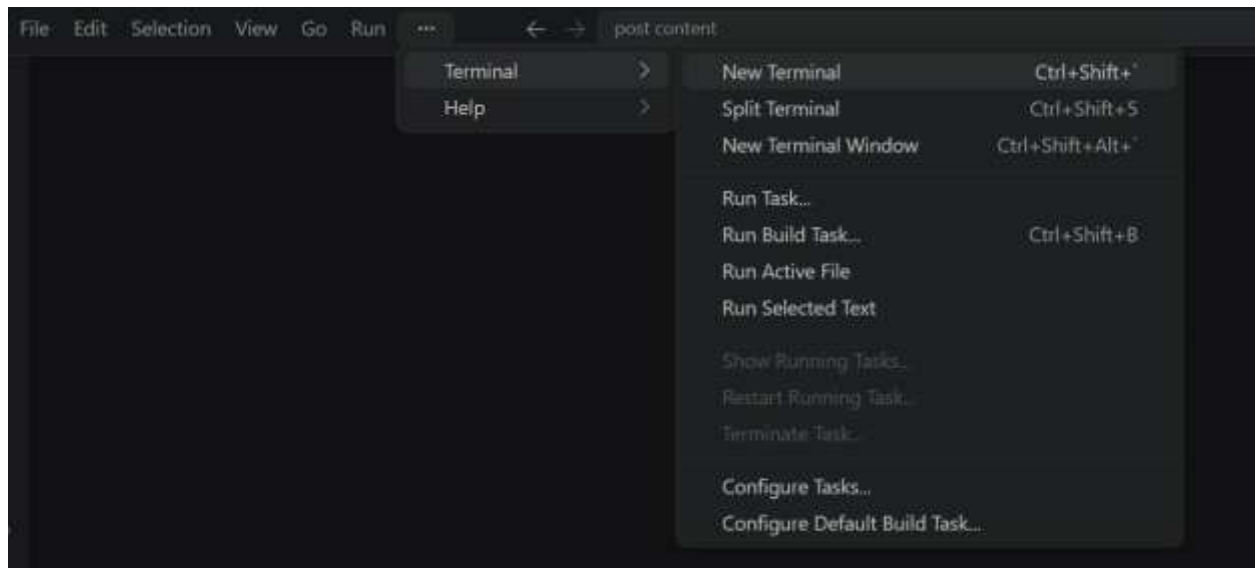
Milestone 5: Deployment

Milestone 5 focuses on deploying the **OptiCrop** application for testing and demonstration. The application is first executed locally to verify its functionality and then prepared for deployment on a cloud platform such as **Render**. This milestone ensures that the machine learning model, Flask backend, and frontend interface work together seamlessly in a production-ready environment.

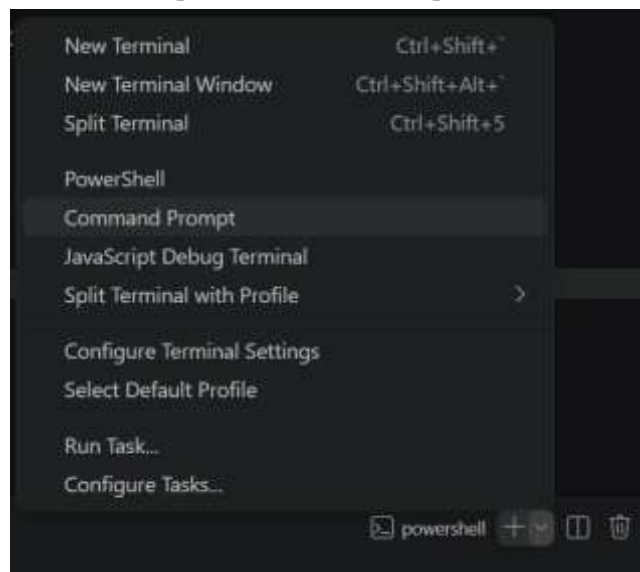
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment

Open **Command Prompt** or **Visual Studio Code Terminal**.



- Then open “Command Prompt”



Create a virtual environment:

```
</> Bash
```

```
python -m venv optienv
```

Activate the virtual environment.

Windows

```
</> Bash
```

```
optienv\Scripts\activate
```

Install all required dependencies.

```
</> Bash
```

```
pip install -r requirements.txt
```

The required libraries include:

- Flask
- Scikit-learn
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Pickle

Configure the Project

Ensure the following files are available in the project directory:

- app.py
- requirements.txt
- crop_recommendation_model.pkl
- scaler.pkl
- dataset.csv
- templates/

- static/
Verify that the trained machine learning model loads correctly before starting the application.

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
Cd OptiCrop
```

```
Python app.py
```

- Once running, open a browser and navigate to “http://127.0.0.1:5050” to access the app.



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG
PS C:\Users\ROHITH\OneDrive\Desktop\SA code> cd OptiCrop
PS C:\Users\ROHITH\OneDrive\Desktop\SA code\OptiCrop> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
[]
```

Test the Crop Recommendation System

Enter sample agricultural values:

Parameter	Sample Value
Nitrogen	90
Phosphorous	42
Potassium	43
Temperature	20.8
Humidity	82
pH	6.5
Rainfall	202

Click **Predict Crop**.

The application processes the input and displays the recommended crop on the webpage.

Repeat testing using different input combinations to verify prediction accuracy and application stability.

Activity 5.3: Deploying the Application

Deploy the OptiCrop application using **Render**.

Prepare the Project

Ensure the following files are included:

- app.py
- requirements.txt
- runtime.txt (optional)
- models/

- templates/
- static/

Upload the complete project to a GitHub repository.

Activity 5.3: Deploying the Application

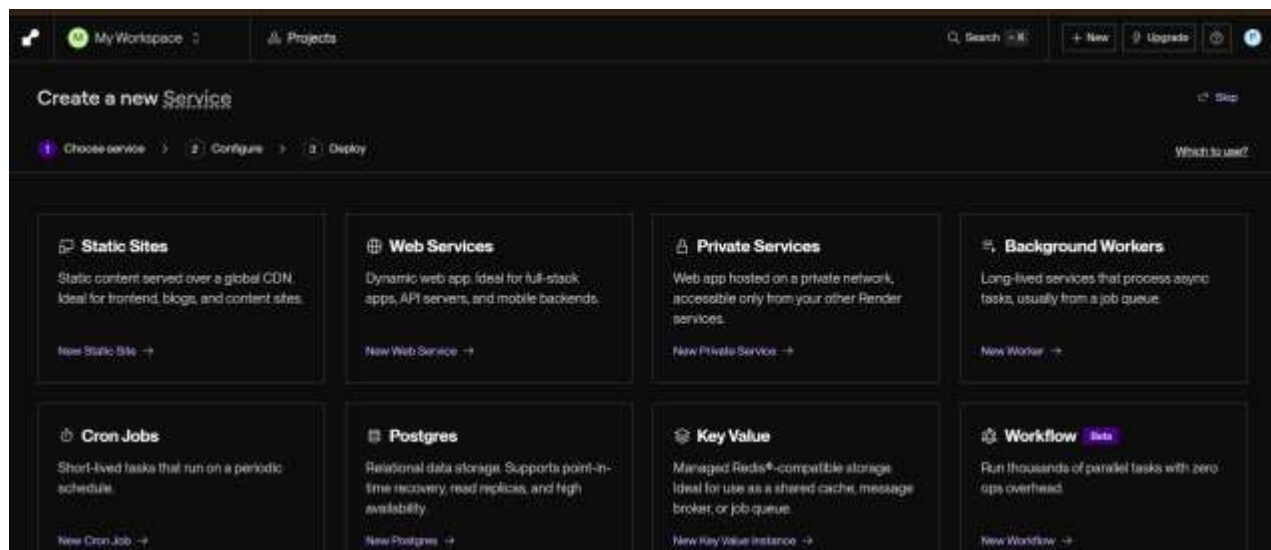
Deploy the OptiCrop application using **Render**.

Prepare the Project

Ensure the following files are included:

- app.py
- requirements.txt
- runtime.txt (optional)
- models/
- templates/
- static/

Upload the complete project to a GitHub repository.



Deploy on Render

1. Sign in to Render.
2. Select **New Web Service**.
3. Connect your GitHub repository.
4. Select the OptiCrop repository.
5. Configure the deployment.

Build Command

```
</> Bash  
  
pip install -r requirements.txt
```

Start Command

```
</> Bash  
  
python app.py
```

Click **Deploy**.

Render automatically builds and deploys the application.

After deployment, Render generates a public URL similar to:

```
https://opticrop.onrender.com
```

The application can now be accessed through any web browser.

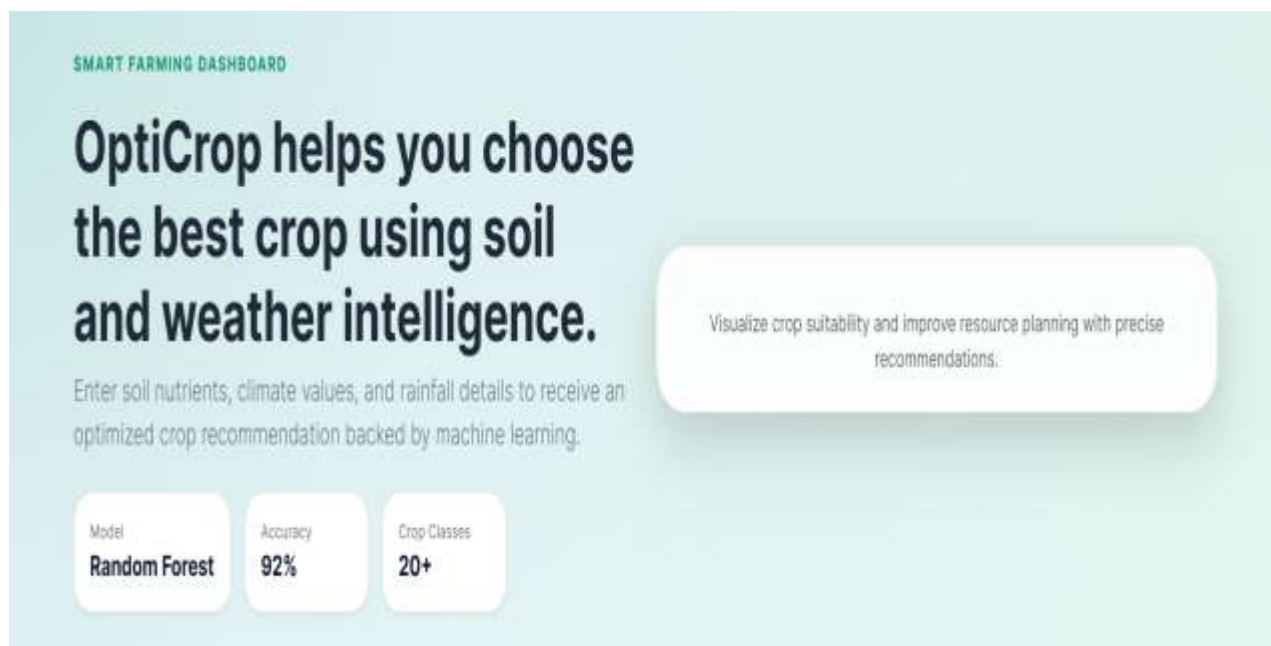
Important Notes

- Ensure all required Python packages are listed in **requirements.txt**.
- Keep the trained model (.pkl file) inside the project directory.

- Verify that all template and static files are uploaded.
- Test the deployed application to confirm that predictions work correctly.
- If deployment fails, review the Render build logs and resolve dependency issues.

Exploring the Website's Web Pages:

Home / Generator Page:



Description : The Home Page serves as the main interface of the OptiCrop application. It provides an overview of the project and displays the machine learning model information, prediction accuracy, and supported crop classes. Users can navigate easily to the crop recommendation section through a clean and responsive dashboard.

Input Section:

Crop Recommendation

Provide input values for precision farming guidance.

N ⓘ

P ⓘ

K ⓘ

Temperature ⓘ

Humidity ⓘ

pH ⓘ

Rainfall ⓘ

Recommend Crop

Live prediction

How OptiCrop Works

- 
Smart preprocessing
 Normalizes soil and weather inputs so the model can compare values accurately.
- 
Machine learning model
 Uses a trained Random Forest model to identify the crop best suited for your conditions.
- 
Reliable decision support
 Provides fast crop guidance for more efficient planning and better yield outcomes.
- 
Adaptive recommendations
 Can be extended with more crop data to improve predictions over time.

Pro tip

For better results, use up-to-date soil test values and local weather data on each prediction run.

Description: The Input Section consists of a clean and responsive card where users enter the agricultural parameters required for crop prediction. The form includes input fields for Nitrogen (N), Phosphorous (P), Potassium (K), Temperature, Humidity, pH, and Rainfall. After entering the values, users click the "Predict Crop" button to submit the data. The Flask backend validates the inputs, preprocesses the values, and forwards them to the trained Random Forest machine learning model for prediction.

Results Section:

Recommendation

Confidence **40.5%**

Chickpea

Best-suited crop based on your soil nutrition and weather values.

Best available Match

This is the best crop for your conditions among available options.

What to fix before planting

- N** N is too low — consider increasing it.
- P** P is too low — consider increasing it.
- K** K is too low — consider increasing it.
- pH** pH is too high — consider reducing it.
- rainfall** rainfall is too high — consider reducing it.

Chickpea **Best**

40.5% confidence

Mango

25.0% confidence

Coconut

15.0% confidence

weather data on each prediction run.

Recent Predictions

Chickpea

N:10.0 P:20.0 K:30.0

40.5%

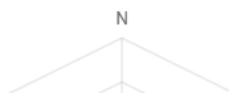
Top 3 Crop Confidence Chart



Your Inputs vs Ideal Conditions

How your soil and climate values compare to the ideal profile for Chickpea.

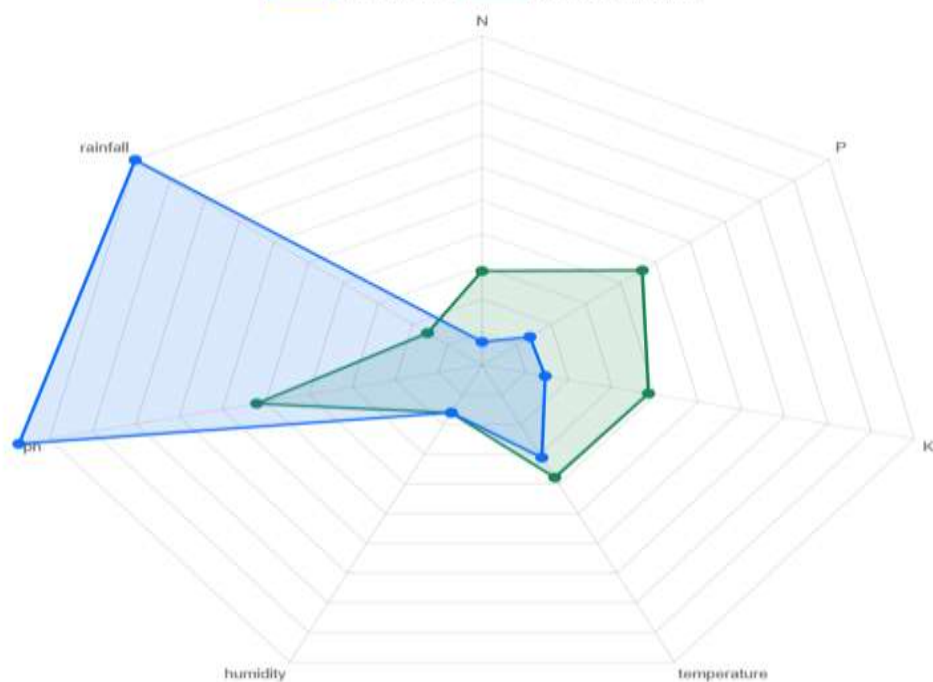
Your Inputs
Ideal for Chickpea



Your Inputs vs Ideal Conditions

How your soil and climate values compare to the ideal profile for Chickpea.

Your Inputs
Ideal for Chickpea



💡 How to read this chart

- The chart has **7 spokes** — one for each soil and weather value you entered (N, P, K, Temperature, Humidity, pH, Rainfall).
- The **green shape** shows the **ideal values** for Chickpea — this is what the crop grows best in.
- The **blue shape** shows **your field values** — what you entered.
- When the **blue and green shapes overlap closely**, your field is a great match for this crop.
- Where the **blue shape is far from green**, that parameter needs attention — consider adjusting fertilizer, irrigation, or waiting for a better season.

N
10.0

P
20.0

K
30.0

TEMPERATURE
14.0

HUMIDITY
16.0

PH
15.0

RAINFALL
300.0

Crop guide

Chickpea growing profile

Chickpea is a drought-tolerant pulse crop that improves soil nitrogen and suits cooler dry seasons.

PH
6.0 - 7.0

TEMPERATURE
10 - 25°C

RAINFALL
35 - 60 mm

TIPS
Plant in well-drained soil

How OptiCrop makes this recommendation

- Validates your soil, climate, and rainfall inputs for consistency and quality.
- Scales the input values so the model can compare all parameters fairly.
- Uses a trained Random Forest model to estimate which crop is best suited to your conditions.
- Delivers a practical recommendation aimed at improving productivity and resource efficiency.

Description: After processing the user inputs, the Results Section dynamically displays the crop prediction along with comprehensive agricultural insights. It presents the recommended crop, prediction confidence percentage, and the top three crop recommendations ranked by confidence. The application also compares the user's soil and weather values with the ideal conditions for the recommended crop using graphical visualizations. Additionally, it provides suggestions for improving soil nutrients or environmental parameters, a crop-growing guide containing ideal pH, temperature, and rainfall ranges, and practical farming tips. The dashboard also explains how the machine learning model generated the recommendation and maintains a history of recent predictions, enabling users to analyze previous results effectively.

Conclusion:

OptiCrop: Smart Agricultural Production Optimization Engine is an intelligent machine learning-based web application designed to assist farmers and agricultural professionals in selecting the most suitable crop based on soil nutrients and environmental conditions. By integrating **Flask**, **Random Forest**, and advanced data preprocessing techniques, the system delivers accurate crop recommendations in real time. The application not only predicts the best crop but also provides confidence scores, alternative crop suggestions, soil improvement recommendations, and visual comparisons with ideal growing conditions. Its user-friendly interface and decision-support features make it a practical solution for improving agricultural productivity, optimizing resource utilization, and promoting sustainable farming practices. In the future, the system can be enhanced with real-time weather integration, IoT sensor connectivity, fertilizer recommendations, multilingual support, and cloud deployment to provide even more intelligent and scalable agricultural solutions.